Q-SHIFT: An Adaptive Quantum Error-Correction Framework for Nonstationary Noise

Integrated research report | evidence through Phase 5E

Abstract

Quantum error-correction systems use repeated measurements to detect faults before they corrupt protected information. A decoder maps these patterns to a likely correction. Most systems use a fixed noise model, although real devices can drift. Q-SHIFT investigates whether recent detector data can guide a choice among decoder models. We developed a Python prototype using Stim circuit simulation, PyMatching matching decoders, a nine-model gate/measurement grid, detector-window estimation, and a causal controller with a pooled-fixed fallback. Condition-specific calibration sometimes improved on a nominal fixed model under measurement-heavy noise. However, Phase 4C compared calibration with a stronger pooled fixed decoder across three regimes and found no overall gain: the equal-regime difference was +0.00001556, its 95% interval crossed zero, and p=0.667. Later adaptive testing remained exploratory; the latest Phase 5E runner did not switch away from its fallback. Q-SHIFT is a working research prototype, not evidence of hardware deployment or reliable adaptive superiority.

1. Introduction and real-world problem

Quantum computers encode information in qubits, which are sensitive to imperfect gates, measurements, and environmental disturbance. Error correction adds redundancy and repeatedly measures checks on groups of qubits. The outcomes, called syndrome or detector data, do not directly identify each fault; a decoder uses their pattern to estimate a correction. When the assumed noise model differs from the current device, a fixed decoder may make less effective choices. Surface codes are studied because they use local checks on a two-dimensional layout [3].

This project asks whether a decoder can respond to changing mixtures of faults using only measured detector information. The motivation is long quantum computation, where conditions may drift during execution. This is a potential application, not a demonstrated device result.

2. Objective and hypothesis

The objective is to build and evaluate a detector-informed system that chooses among matching decoders as noise changes. The hypothesis is that a causal estimator can identify a useful change early enough for a specialized decoder to lower residual logical errors relative to a strong fixed baseline. Classifying noise is not enough: candidate decoders must produce different corrections that reliably reduce errors.

3. Q-SHIFT system architecture

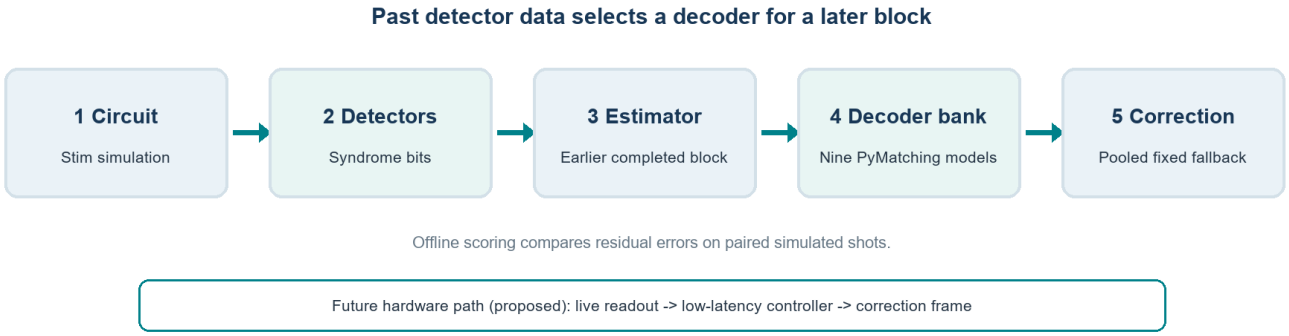


Figure 1. Completed detector blocks inform a decoder choice for a later block. Labels are used only for offline scoring. The hardware interface shown is proposed, not implemented.

Stim generates a rotated surface-code Z-memory circuit and samples gate and measurement faults [1]. It records detector outcomes and one logical observable. PyMatching builds a matching decoder from a Stim detector error model (DEM), a graph-like summary of how faults create detection events [2]. The decoder bank contains nine gate/measurement probability pairs. A pooled fixed decoder remains the fallback.

4. Implementation and experimental method

The main surface-code studies use distance 3 and five measurement rounds. Native Stim circuit-level depolarizing gate noise and basis-appropriate measurement flips are varied across data-heavy (gate 0.006, measurement 0.001), nominal (0.003, 0.003), and measurement-heavy (0.001, 0.006) regimes. These are probabilities per channel invocation, not equal total circuit-fault budgets. Circuit structure and detector/observable definitions remain consistent.

Calibration selects among nine DEM models using detector data only. The Phase 4 method smooths template detector marginals, then scores calibration counts with a composite Bernoulli likelihood and deterministic grid-order tie-breaking. Logical labels are reserved for validation and test scoring. A residual logical error occurs when the sampled observable flip differs from the predicted correction. Paired comparisons give all decoders identical detector shots and labels.

For dynamic development, an estimator compares recent completed detector windows with stored profiles. The controller includes confidence, hysteresis, dwell-time, switch-penalty, and fixed-fallback options. It chooses for the next block using earlier detector data. Hidden-regime and hindsight comparisons are privileged diagnostics, not deployable policies. No confirmatory adaptive test has run.

5. Chronology of completed research

Phase	Work	Evidence and interpretation
Starter audit	Legacy eight-bit detector set	150,000 rows had only 61 unique patterns in the audit. Repeated rows undermine random splits; early classifier/XOR metrics are not validated decoder evidence.
Phase 1	Repetition-code regimes and recognition	Recognition was reported at 94.4%, but guarded adaptation made zero switches and tied fixed matching. Per-regime report/JSON totals conflict.
Phase 2	Composition-specific repetition-code matchers	Ten seeds, six conditions; validation selected fixed matching in all 60 choices. No specialist gate passed.
Phase 3A / 3B	Surface-code pilot and replication	Pilot signal failed multiplicity control; independent 3B test found 43,675 specialist vs 43,673 fixed errors.
Phase 4A / 4B	Circuit-level gate and measurement noise	Static calibration improved measurement-heavy results vs nominal fixed. 4B generated fresh templates contrary to written reuse protocol.
Phase 4C	Baseline selection and held-out test	1.2M development shots froze baselines; 1.8M Stage 2 shots found no overall calibrated gain vs pooled fixed.
Phase 5 v3	Seven dynamic development schedules	61,440 paired shots; utility and ML policies were worse in aggregate than pooled fixed.
Phase 5E audit	Decoder specialization feasibility	Nine graph signatures differ. Executed source/JSON: four families x three repeats; adaptive made zero switches.

What the process changed

The project moved from a repetitive eight-bit repetition-code dataset toward controlled surface-code circuits with explicit detector and logical-observable definitions. It replaced accuracy-focused classification with residual logical-error comparisons, paired shot-level differences, and seed-level uncertainty. Baselines also became stronger: nominal matching is informative, but a pooled decoder selected on development data is a harder test.

Phase 3A v1 used an invalid probability interpretation and is excluded. Corrected v2-v4 reused the same seed bundles and form one pilot, not three independent replications. Phase 3B is the fresh replication. Phase 4C Stage 1 selected baseline models on development samples before Stage 2 testing.

6. Verified experimental results

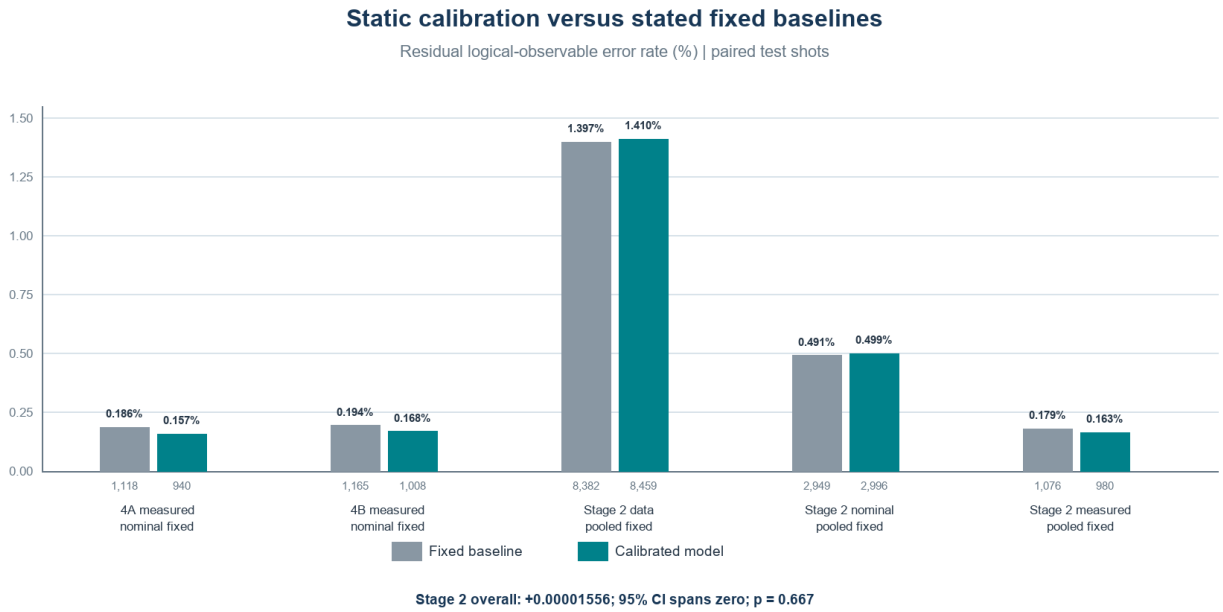


Figure 2. Residual error rates. Phase 4A/4B compare with nominal fixed; Stage 2 uses stronger pooled fixed. The Stage 2 interval appears in the table.

Study	Test design	Errors / key statistic	Interpretation
Phase 3B specialist vs fixed	600k paired; 20 roots	43,675 vs 43,673; p=.9374	No replicated gain.
Phase 4A calibrated vs nominal	600k measurement-heavy	1,118 vs 940; discordances 80/258; p=5.60e-23	Static condition-specific gain.
Phase 4B calibrated vs nominal	600k; 20 roots	1,165 vs 1,008; discordances 99/256; p=3.73e-17	Repeat; fresh-template deviation.
Phase 4C calibrated vs pooled	1.8M; 60 roots	12,435 vs 12,407; difference=+.00001556; CI [-.00004889,+.00007667]; p=.6666	No overall gain.

Stage 2 calibrated-minus-pooled errors were +77 data-heavy, +47 nominal, and -96 measurement-heavy. The net +28 errors over three equal-sized regimes gives the near-zero primary difference. Its 100,000-draw paired root-cluster sign-flip test gave p=.66658; the regime-stratified seed-bootstrap 95% interval included zero. This is not evidence of an overall gain.

Phases 4A and 4B support a narrower finding: calibrated static models reduced errors relative to a nominal model in measurement-heavy noise. They do not test online switching and do not establish superiority over pooled fixed. Phase 4B predictions were audited, but template generation departed from protocol.

7. Adaptive development and decoder specialization

Phase 5 v3 evaluated seven schedules: stationary, two transition directions, a return-to-nominal sequence, slow, rapid, and random changes. Across 61,440 development shots, utility selection made 435 errors versus 425 pooled; maximum-likelihood selection made 441. Results varied by schedule. These small development streams are not independent tests. Source and tests implement causal ordering, hysteresis, and fallback, but do not demonstrate a gain.

The Phase 5E saved JSON and runner show 30,000 disagreement shots per regime (90,000 total), 45,000 calibration shots, 90,000 stationary evaluation shots, and 36,864 dynamic shots: four families, three repeats, twelve blocks, 256 shots per block. Across twelve streams, both adaptive-labelled choices stayed at pooled decoder #1, made zero switches, and tied pooled errors exactly (298 each). The utility-labelled choice is the same code path as the ML choice in this runner.

The Phase 5E log instead says seven families, four repetitions, and 90,000 disagreement shots per regime. Those statements conflict with source loops and JSON and are not used. Regime-matched and calibrated-regime comparators use hidden state; hindsight uses current-block labels. These are privileged diagnostics. Nine DEM/weight signatures differ, but four model pairs had no disagreements in each sampled 30,000-shot regime.

8. Achievements and contribution

The project delivered a Stim simulation, a nine-model PyMatching bank, detector-only calibration, causal estimator/controller components, paired evaluation scripts, seed/checkpoint records, and independent audits of selected test predictions. It learned to distinguish classifier accuracy from residual logical-error reduction and to separate development, validation, and test evidence. Models with different weights can produce the same correction; different corrections do not guarantee a practical improvement. This is a useful feasibility finding, not a negative verdict on all adaptive decoding.

9. Proposed real-world implementation

A hardware pilot could receive syndrome outcomes from each correction cycle, maintain a rolling window, and select among prebuilt decoders for later cycles. A validated fixed decoder would remain the fallback; switching would require sustained evidence and be logged. A correction could be returned as a Pauli-frame update or an equivalent instruction. This interface is proposed only.

Deployment would require real-device readout, device-specific calibration, validation on held-out times and qubits, drift and unfamiliar-noise safeguards, and a processor-adjacent CPU/FPGA/accelerator implementation. It also needs deterministic latency guarantees, safe fallback behavior, reliable data routing, and end-to-end timing measurements. A hardware trial must compare fixed and adaptive methods on matched runs and count calibration overhead.

10. Potential applications and impact

If later proven reliable, adaptation could help protect logical information during longer computations. Better error correction may support more dependable quantum simulation in chemistry or materials research and reduce wasted processor time. These are downstream possibilities, not project results.

11. Limitations and future work

Evidence covers one small distance-3 Z-memory circuit, selected Pauli-noise models, and simulation. It does not cover hardware drift, leakage, correlated/burst faults, larger code distances, or live control. Some early tables conflict; disputed Phase 1 totals and the Phase 2 one-count oracle aggregate are not used as exact claims. Phase 4B has a template deviation. Stage 2 predictions were independently re-decoded, but calibration samples were not retained, so fitting cannot be regenerated. Phase 5E narrative counts conflict with runner/JSON. Future work should preserve calibration arrays, version analysis, use multiple independent schedule seeds, and freeze a policy before fresh held-out dynamic testing.

12. Conclusion

Q-SHIFT is a functioning simulation research prototype combining noise estimation, multiple matching decoders, and causal selection. Experiments show condition-specific static gains against a nominal model, but no reliable adaptive advantage against a strong pooled baseline. Its contribution is a transparent platform for testing when adaptation may help. Hardware benefit remains open.

References

[1] C. Gidney, "Stim: a fast stabilizer circuit simulator," Quantum 5, 497 (2021). DOI 10.22331/Q-2021-07-06-497.

[2] O. Higgott, "PyMatching: A Python Package for Decoding Quantum Codes with Minimum-Weight Perfect Matching," ACM Transactions on Quantum Computing 3(3), Article 16 (2022). DOI 10.1145/3505637.

[3] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, "Surface codes: Towards practical large-scale quantum computation," Physical Review A 86, 032324 (2012). DOI 10.1103/PhysRevA.86.032324.